

SupportFlow

A text-AI design, implementation and evaluation plan

One. Introduction.

Implementation repository: <https://github.com/happyhnyb/supportflow>

Customer support teams throughout the world are some of the busiest teams in the entire businesses, especially if you talk about developing countries, big billion-dollar family businesses are super common. And among these family businesses, customer care becomes the most difficult and tedious task as no one in the family wants to do this. So they have to outsource it, even before starting anything.

The customer support agent, in bracket AI slash human, needs to follow a SOP, which is note down all the issues such as late deliveries, refunds, damaged products, and payment problems. Then they assess it by how urgent it is. For example, if someone has not received his or her delivery since two weeks, it is a problem worth escalating. And unfortunately, the customer support teams throughout the world are not very efficient at doing that.

So what I want to prove with my project is that if we make a really good AI-based agent, it can take over the whole customer support ticket system and run it smoothly.

The name which I have given to my product is Support Flow. Now Support Flow is an AI agent which has been made to assist customer support, and it does that by identifying the customer issues, estimate the confidence in the prediction it is giving out, identify which ticket is the most urgent, remove most of the useless information from the texts, because some customers send long texts with baseless information, which my agent doesn't even need.

Identify the order numbers and make a summary for the support agent in bracket human.

What this does is it becomes the main brains of it, but it doesn't totally replace the customer care executor, as that would lead to unemployment, and AI was not made to do that. Because humans might be super inefficient at the job they are doing, and they probably should be replaced for more efficiency. But unfortunately, we are not evil enough to do that.

Part Two: Project Objectives.

The main objective of this project is to demonstrate that how we can use artificial intelligence and machine learning models together and make a practical, useful product with them.

The main objectives are:

- make a machine learning model for segregating, classifying, and understanding the customer support message;
- two, remove any sensitive information such as phone number, address, and then pass it through the system;
- number three, identify important information such as order number automatically;
- number four, pull out the SOP knowledge from the database with context to current ticket;
- five, send the information using FastAPI to a web interface;
- six, use generative AI model in order to guide the human agent;
- seven, keep human agent responsible for final action.

This would tell the human what to do, and it won't do that, so no chaos would be caused.

So therefore, I say that the project focuses not just on making models or using ML models and AI together, but also on demonstrating how this pair could work in real industries.

Part Three: System Overview.

Multiple components come together in order to make system flow work:

customer message, arrow, data validation, arrow, TII redaction, arrow, machine learning models, arrow, classification, arrow, knowledge retrieval, arrow, optional AI review, arrow, urgency detection, arrow, agent dashboard.

The customer message first goes through the browser, then FastAPI backend makes sure of the input and gets rid of sensitive information. The message which comes out of this pipeline is passed through machine learning models, which predict its category.

When configured, the server-side OpenAI review uses `gpt-5-mini` by default to reconsider the classifier probabilities and prepare structured guidance. The application then presents the result to a human agent for action.

Number four, dataset.

The model uses a reproducible 600-row subset of Bitext's public Retail (eCommerce) LLM Chatbot Training Dataset. The marker can access the original 44,884-row dataset at <https://huggingface.co/datasets/bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset>

It is published under the CDLA-Sharing-1.0 licence.

The script `scripts/prepare_dataset.py` downloads a checksum-pinned source revision and selects 100 examples from each of six source intents with random seed 42. It maps `delivery_issue`, `request_refund`, `damaged_delivery`, `payment_issue`, `recover_password`, and `change_order` to SupportFlow's six operational categories. The derived CSV retains the source intent and source row number for traceability. Full provenance, the exact revision, checksum, mapping, licence, and rebuild command are in `data/README.md`.

The subset is balanced, which makes the small evaluation easier to interpret and prevents one label from dominating training. However, it is generated conversational data rather than real customer records. It is suitable for demonstrating the pipeline, not for making claims about production performance.

This is what a real system should have: an accurate system where the tickets go into the right section, so no further delays can take place and the human agent exactly knows what needs to be done.

Part five: data preprocessing and privacy.

Before sending any data forward, system flow makes sure that no sensitive information is leaked. Therefore, it gets rid of all the sensitive information using a Python script, which could be seen in `privacy.py`.

This script gets rid of data such as email addresses, telephone numbers, and payment information, and it puts fields such as email redacted, phone redacted, and card redacted.

The system also recognizes order numbers. For example, order AB # 123, and it would normalize this to an order ID, which is AB 123, which is much easier for the machine learning model to understand. So therefore no confusion is caused at any point in the whole workflow.

Machine learning model.

The main machine learning model which we are using over here is TF-IDF vectorization and logistic regression.

TF-IDF.

Machine learning algorithms don't work like human brains, so it is impossible for them to understand string sentences. String sentences are the sentences which humans talk in, which is plain English. They need numbers and weights in order to understand that.

TF-IDF does the exact same thing. It changes the words into weights or numbers.

6.2, logistic regression.

The TF-IDF features are then passed through logistic regression classifier. This is where the magic happens. This classifying model can actually find out the weights of each ticket and it puts it accordingly in our categories, such as delivery delay, refund request, damaged item, payment problem, and account access, and order charge.

After this category, most of the work is done as weights are given to it, and now we are having insights. Now comes time to turn these insights into actual outcomes for the human agents.

Part 7: Model Training.

The training for these models take place in train.py. So we have used a mixture of 80% training data and 20% testing data. This makes sure that there is no homogeneous or heterogeneous categories, and the data is properly split between the two categories.

The random state is fixed at 42, which means that we can reproduce this experiment again and again by getting the same result.

In the beginning, the model is only trained with training data, and then it is evaluated by comparing it with the testing data. After the evaluation is done and we are satisfied, one more model is trained using all the labeled data.

Now, after this we get the final model, and this final model is saved using joblib, which is a library, and then it is used in the application.

This is very important because if we are making a machine learning model, we need to make sure that it is highly reproducible, which means when we run the experiment and training again and again, same outcomes come out.

Part 8: Model Evaluation.

There are two scores which we are using over here. One happens to be accuracy, and the second one happens to be Macro F1 score.

The reproducible run achieved 1.000 accuracy and 1.000 Macro F1 on the 120-row stratified holdout. These perfect scores must be interpreted cautiously rather than treated as evidence of a perfect real-world model.

Macro F1 is very useful for us because it actually calculates the performance which our model is giving out. This would make a benchmark and make sure commonly occurring class does not get to dominate the overall result.

The evaluation uses 120 held-out examples, or 20 percent of the 600-row subset. These results demonstrate that the training and evaluation pipeline works, but they do not establish production readiness. The examples come from one public generated dataset, and random splitting may place closely worded utterances in both partitions. A production evaluation should use unseen, time-separated, human-reviewed tickets from the deployment context.

Larger real-world datasets may improve the model, but performance would need to be demonstrated on representative unseen tickets and monitored after deployment.

Urgency Detection.

So, in order to understand urgency detection, what we have to know is that ticket category and ticket urgency are not the same problems. They are two different problems. One tells what kind of category the problem is, and one tells when should the problem be solved.

Of course, people who are having complaints in last seven days should get their stuff resolved before the one who just gave it.

When a customer raises a ticket such as, I was charged two times and I need it investigated urgently, this would tell the system that the customer cannot afford to lose times, and therefore this goes down as an urgent request.

And these two are kept separate because if they were together, then classification and urgency would collide, which means if model is very certain that this is a case of late delivery, it would also call it urgent, no matter if it was urgent or not.

Part 10: Knowledge Base Retrieval.

The system contains a small SOP knowledge base. SOP happens to be Standard Operating Procedure, and it is in `knowledge_base.json`. There are in total seven articles and six categories.

The examples include late deliveries and tracking policy, returns and refund eligibility, damaged item replacement procedure, payment failure duplicate charge guide, account access and password reset.

The knowledge base again uses TF-IDF. When a message is analyzed, the knowledge base is searched for something which matches the semanticness of it, or the similarity of it, and if similarity is matched with another incident which has happened in the past, knowledge of that is pulled out and weights are re-given.

So therefore a better decision could be taken, as metrics from the past are also now taken into consideration.

11. Generative AI integration.

Generative AI integration has been kept as an optional layer over here, but I would say it is one of the most crucial features. So it is better if it is used. But it is a bit of a cost-heavy feature; therefore, it's optional.

Machine learning insights are enough to tell a smart customer agent what needs to be done, as it is going to do the categorization, it is going to tell him the urgency. But if he needs proper instructions, what needs to be done, then we would have to use generative AI over here.

The AI is given customer ticket, probabilities made by local classifier, and relevant data from knowledge base as input data, which is then used to produce intent, urgency, summary, and the next step, as well as rationale.

The application is expected to follow a pre-existing JSON schema, and there are guardrails baked into it that would make sure refunds are not given just like that, or delivery dates, or replacement, as well as any action.

If we have not put an API key in the env file, the AI service will not work, and support flow will simply rely on machine learning models in order to produce and create tickets without generative AI.

12. Backend implementation.

So the middleware or web service which we are using over here is FastAPI, and the main endpoint happens to be `POST slash v1 slash tickets slash analyze`.

The request body for API is expected to be a customer message, and the message can be between 3 and 5,000 characters.

Once this message is sent, the payload which we get in return is redacted customer message, predicted intent, readable intent name, confidence, probability for every category, urgency, urgency explanations, detected order IDs, internal summary, recommended next step, matching knowledge articles, as well as information on if the output is from baseline or from an AI review.

A health route also happens to exist to check if the service is working properly or not.

13. User interface.

Well, user interface is not very complicated. It is just a browser-based dashboard, and it is made using index HTML, style CSS, and script JS.

This interface makes the support agent paste a customer message and select the review ticket, and result is displayed over there after compute takes place.

The browser communicates with server using FastAPI, using JS's fetch function.

And note, it is just an assistant tool. So the service cannot take decision itself and go and send customer refunds or make any promises to the customers. It can just tell the human agent what needs to be done.

14. Testing.

This project has many automatically running tests using pytest. These tests do everything, from checking privacy to order extraction functionality, by confirming that email addresses and phone numbers are removed once it is detected in the text given by human agent.

Another test runs the trained model with the SOP knowledge base about broken headphones and then categorizes it as damaged item and matches with appropriate damaged item knowledge article.

These tests just are not there for the sake of project, but these tests tell us that the service is actually working, and once put in real industries, it is not going to cause any errors but is going to work.

15. Limitations.

This is a university project, so it cannot be treated as a production-grade product. The largest limitation is external validity: the 600 selected messages are generated English utterances from one public dataset and cannot represent the language, products, policies, or edge cases of a real support operation.

Then comes limitation number two. Often customers just don't give out one problem. Sometimes they have multiple problems, for example, receiving a product late, which was found to be broken. The service does not take problems like that into consideration. It is just single-problem-based system.

Limitation number three. The privacy system works very well in tests, but I have no idea if it is going to work on a scale or not. That is, with multiple messages. So, therefore, I cannot confirm if it is going to be working as a real product or not.

Limitation four. Urgency detection system just works on keywords and could generate false positives as well as false negatives.

Fifth and last limitation is that knowledge base is not that diverse. That's why multiple knowledge points cannot be established from that. So there are still chances that a problem comes in which has never been had, and the agent or the service goes blank and has no idea what needs to be done over here.

Sixteen. Future improvements.

Well, if we want to improve this project, sky is the real limit here.

The first improvement would be to make sure that we have a real and a ginormous dataset so the models can be trained better, and we know if scripts are working in bulk or not, and if it can handle parallel workflows or not.

Additional techniques should also be put here, such as confusion matrices, precision and recall, chronological train/test splits, confidence calibration, as well as cross-validation.

We could also use advanced language models, which are transformer-based, such as Claude Opus 5 or Claude Fable 5.1, and then compare the results with logistic regression.

Seventeen. Conclusion.

So the support flow is not a real production-grade app right now. It's just a university project, and if we talk about in real life, it is a POC, or proof of concept, which means it proves to us that something like this would work in the market, and it could solve real business problems and give insights which would help real businesses go further and be more efficient.

Being a part of someone who has a family business, I think something like this would ease a lot as instead of hiring five or six people just for customer support, there needs to be two or three of them with a technology like this.

And with further development and widening of horizons, this can turn into a real product.

references

FastAPI (2026). FastAPI Documentation.

OpenAI (2026). OpenAI API Documentation.

Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python". Journal of Machine Learning Research, 12, pp. 2825–2830.

Python Software Foundation (2026). Python Documentation.

Scikit-learn Developers (2026). Scikit-learn User Guide.

Bitext (2024). Retail (eCommerce) LLM Chatbot Training Dataset.
<https://huggingface.co/datasets/bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset>

SupportFlow source code (2026). <https://github.com/happyhnyb/supportflow>